



UNIVERSIDAD
TECNOLÓGICA
DE PUEBLA

UNIVERSIDAD TECNOLÓGICA DE PUEBLA

**INGENIERÍA EN DESARROLLO DE SOFTWARE
MULTIPLATAFORMA**

APLICACIÓN NATIVA

REPORTE — SUBPRODUCTO NO. 1

ASIGNATURA: APLICACIONES WEB PROGRESIVAS

DOCENTE: MATHER XÓCHITL MENDOZA PÍSCIL

ALUMNO: PÉREZ ALCÁNTARA ALEJANDRO 10° “D”

FECHA: 6 DE SEPTIEMBRE DE 2026

ÍNDICE

1. Introducción.....	2
2. ¿Qué es una Aplicación Nativa?.....	2
3. Ilustración: ¿Cómo funciona una Aplicación Nativa?	2
4. Características Principales	3
5. Ventajas y Desventajas	4
6. Comparativa: Nativa vs. Web vs. Híbrida	4
7. Tecnología Utilizada.....	5
8. Mi Ejemplo De Aplicación Nativa: Proyecto Wear Os	5
8.1 Objetivo Del Proyecto	5
8.3 Parte 1 — Alerta Pediátrica Wear OS (Teléfono → Reloj)	5
8.5 Código O URL De La Aplicación	6
9. Imágenes De Mi Aplicación	7
9.1 Alerta Pediátrica: Del Teléfono Al Reloj.....	7
9.2 Menú Wearable / APA-Menú Wearable Pro: La App Técnica Del Reloj	10
10. Ejemplo De Una Aplicación Nativa Reconocida: Spotify	20
11. Lista de Cotejo.....	21
12. Conclusión	23
13. Referencias.....	24

1. Introducción

En el desarrollo de software para dispositivos móviles existen distintos enfoques de construcción de aplicaciones: nativo, web e híbrido. En este reporte me enfoco en las aplicaciones nativas, un tipo de desarrollo que sigue siendo el estándar de la industria cuando se busca el máximo rendimiento y la integración más completa con el hardware y el sistema operativo del dispositivo.

A lo largo del documento defino el concepto, ilustro su funcionamiento, listo sus características principales, sus ventajas y desventajas, lo comparo frente a los enfoques web e híbrido, documento la tecnología que se usa para construirlo, presento el ejemplo real de una aplicación nativa ampliamente utilizada (Spotify) y, finalmente, documento mi propio ejemplo de aplicación nativa: el proyecto de reloj inteligente (Wear OS) que desarrollé el cuatrimestre pasado en la materia de Desarrollo para Dispositivos Inteligentes, con la maestra Jenny Ortega García.

2. ¿Qué es una Aplicación Nativa?

Una aplicación nativa es un programa de software desarrollado específicamente para un sistema operativo determinado (iOS o Android), utilizando los lenguajes de programación, frameworks y herramientas oficiales de esa plataforma. A diferencia de una aplicación web —que se ejecuta dentro de un navegador y funciona igual en cualquier sistema operativo—, una aplicación nativa se instala directamente en el dispositivo a través de una tienda de aplicaciones (App Store o Google Play) y se compila para ejecutarse de forma directa sobre el sistema operativo, lo que le permite acceder por completo al hardware y a las funciones propias del dispositivo.

Este enfoque no se limita a teléfonos: también existe desarrollo nativo para relojes inteligentes (Wear OS, watchOS), televisiones (Android TV, tvOS) y otros dispositivos con sistema operativo propio, cada uno con su propio SDK y sus propias reglas de diseño. En la sección 8 de este documento explico, con evidencia propia, cómo apliqué exactamente este enfoque al desarrollar una app nativa para Wear OS.

3. Ilustración: ¿Cómo funciona una Aplicación Nativa?

El siguiente diagrama ilustra el flujo general de una aplicación nativa, desde su distribución en la tienda de aplicaciones hasta su ejecución directa sobre el hardware del dispositivo, pasando por las APIs oficiales que ofrece cada sistema operativo.

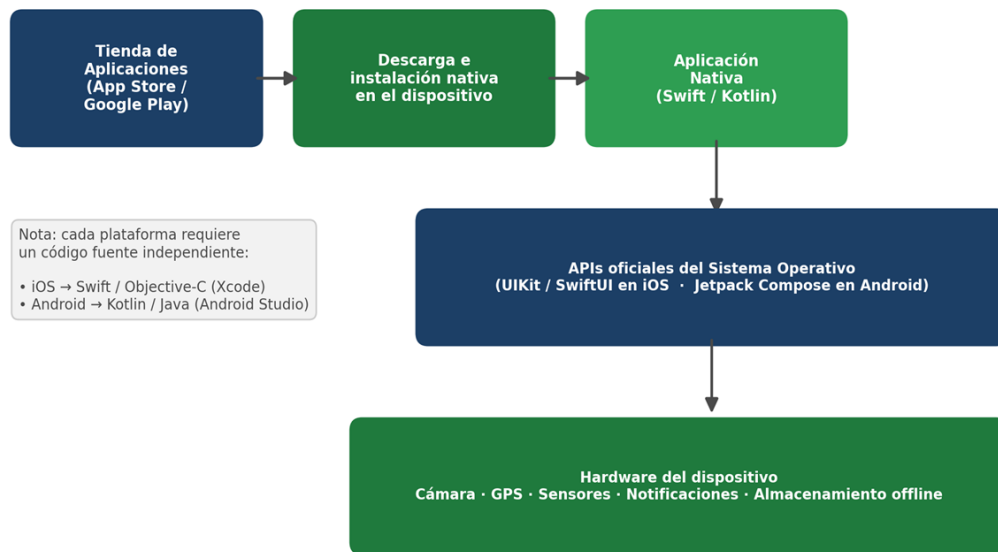


Figura 1. Flujo de funcionamiento de una aplicación nativa, desde la tienda de aplicaciones hasta el hardware del dispositivo.

4. Características Principales

- Rendimiento optimizado: al compilarse directamente para el sistema operativo, sin capas intermedias, ofrece los tiempos de respuesta más rápidos y el uso más eficiente de recursos.
- Acceso total al hardware: puede utilizar cámara, GPS, sensores, Bluetooth, huella digital, reconocimiento facial y otras funciones específicas del dispositivo.
- Funcionalidades avanzadas: permite implementar notificaciones push, comunicación directa entre teléfono y wearable, realidad aumentada y procesamiento en segundo plano.
- Funcionamiento sin conexión: puede ejecutar funciones clave y almacenar datos localmente aunque el dispositivo no tenga acceso a internet.
- Distribución mediante tiendas oficiales: se instala a través de App Store o Google Play, lo que implica un proceso de revisión y aprobación antes de publicarse.
- Desarrollo específico por plataforma: requiere un código fuente independiente para iOS y otro para Android (y, en el caso de wearables, un módulo adicional para el propio reloj), lo cual incrementa el tiempo y costo de desarrollo frente a otros enfoques.

5. Ventajas y Desventajas

A partir de las características anteriores, resumo las principales ventajas y desventajas del desarrollo nativo frente a otros enfoques:

VENTAJAS	DESVENTAJAS
Máximo rendimiento y fluidez, al ejecutarse directamente sobre el sistema operativo.	Requiere mantener dos (o más) bases de código independientes: una para iOS y otra para Android.
Acceso completo al hardware: cámara, GPS, sensores, notificaciones push, Bluetooth hacia wearables, etc.	Mayor tiempo y costo de desarrollo en comparación con soluciones web o híbridas.
Mejor experiencia de usuario (UI/UX), apegada a los estándares visuales de cada plataforma.	Cada actualización debe pasar por el proceso de revisión y aprobación de la tienda.
Funcionamiento sin conexión a internet para funciones clave.	Requiere conocimientos especializados en cada lenguaje y plataforma (Swift/Kotlin).
Mayor seguridad, al pasar por un proceso de revisión de la tienda oficial antes de publicarse.	El usuario debe descargar e instalar la app antes de poder usarla.

6. Comparativa: Nativa vs. Web vs. Híbrida

Para dimensionar mejor a la aplicación nativa dentro del panorama general del desarrollo de aplicaciones, la comparo frente a los otros dos enfoques principales: la aplicación web y la aplicación híbrida.

CRITERIO	NATIVA	WEB	HÍBRIDA
Lenguaje	Swift/Kotlin (por plataforma)	HTML, CSS, JavaScript	HTML/JS + wrapper (Flutter, React Native, Ionic)
Rendimiento	Alto	Medio-bajo	Medio
Acceso al hardware	Completo	Limitado (APIs del navegador)	Amplio, vía plugins/puentes nativos
Instalación	Tienda oficial (App Store/Google Play)	No requiere, se accede por URL	Tienda oficial
Funciona sin internet	Sí	Muy limitado	Sí, parcialmente
Costo de desarrollo	Alto (2+ códigos independientes)	Bajo (1 solo código)	Medio (1 código compartido)
Ejemplos	Spotify, Instagram, mi app Wear OS	Aplicaciones web progresivas (PWA), Google Docs	Uber Eats, versiones iniciales de Instagram

La aplicación nativa es la opción con mejor rendimiento y mayor acceso al hardware, pero también la de mayor costo de desarrollo, al requerir un código fuente independiente por cada plataforma.

7. Tecnología Utilizada

El desarrollo nativo se realiza con el lenguaje de programación, el entorno de desarrollo (IDE) y el framework de interfaz oficiales de cada plataforma:

PLATAFORMA	LENGUAJE	IDE OFICIAL	FRAMEWORK DE INTERFAZ
iOS	Swift / Objective-C	Xcode	SwiftUI / UIKit
Android	Kotlin / Java	Android Studio	Jetpack Compose / Views
Wear OS (Android)	Kotlin	Android Studio	Jetpack Compose para Wear OS

8. Mi Ejemplo De Aplicación Nativa: Proyecto Wear Os

Como evidencia propia de desarrollo nativo, retomo en este documento el proyecto que Farid Isaí y yo construimos el cuatrimestre pasado en la materia de Desarrollo para Dispositivos Inteligentes, con la maestra Jenny Ortega García, usando Android Studio y el emulador de Wear OS. A diferencia del resto de PediatriTrack Global (que es una app multiplataforma hecha con React Native), este proyecto es 100% nativo de Android/Wear OS: está escrito directamente en Kotlin y usa Jetpack Compose, el framework de interfaz oficial de Google para construir pantallas nativas modernas.

8.1 Objetivo Del Proyecto

El proyecto se dividió en dos partes que se complementan entre sí: una app de teléfono capaz de enviar notificaciones clínicas directamente al reloj inteligente (pensada como una extensión real de PediatriTrack Global, para que un tutor reciba alertas pediátricas aunque no tenga el teléfono en la mano), y una app técnica interactiva para el propio reloj, pensada para demostrar en Wear OS los temas centrales de la materia: sintaxis de Kotlin, condicionales y ciclos, funciones, manejo de errores con try-catch, y elementos gráficos de Jetpack Compose (cards, botones, listas).

8.2 Herramientas Y Entorno De Desarrollo

Ide: Android Studio

Lenguaje: Kotlin

Framework De Interfaz: Jetpack Compose (Para Wear Os)

Emuladores Usados: Wear Os Large Round Y Wear Os Square

Dispositivo Físico De Prueba: Huawei Watch Gt 6 (Emparejado Por Bluetooth)

Comunicación Teléfono-Reloj: Api De Notificaciones De Android, Sincronizada Hacia El Companion Device Wear Os

8.3 Parte 1 — Alerta Pediátrica Wear OS (Teléfono → Reloj)

Esta primera parte simula un caso de uso real de PediatriTrack Global: desde una pantalla del teléfono capturo un título y un mensaje clínico (por ejemplo, un recordatorio de vacuna) y, al presionar "Enviar alerta al reloj", la app dispara una notificación nativa de Android que se sincroniza automáticamente hacia el reloj Wear OS emparejado, sin que el usuario tenga que hacer nada más en el reloj: la notificación simplemente aparece.

Durante las pruebas documenté tanto el caso exitoso (la notificación llega al reloj y se puede ver, atender o descartar desde ahí) como un caso de error real que me encontré: cuando el reloj no está emparejado o Bluetooth pierde la conexión, la app lo detecta y muestra "Error al buscar reloj Wear OS" en vez de fallar en silencio, lo mismo que hago en el resto de PediatrITrack Global (nunca dejar que la app truene sin explicación).

- Pantalla del teléfono con formulario de título y mensaje clínico.
- Envío exitoso de la alerta y confirmación en pantalla.
- Recepción real de la notificación en el reloj físico Huawei Watch GT 6.
- Manejo del error cuando el reloj no se encuentra disponible.

8.4 Parte 2 — Menú Wearable / APA-Menú Wearable Pro (App Técnica Del Reloj)

La segunda parte es una app que corre directamente en el reloj (probada en los emuladores Wear OS Large Round y Square) y funciona como un menú técnico interactivo: lista los temas vistos en la materia (emuladores aplicables, herramientas de desarrollo, elementos gráficos, sintaxis de Kotlin, condicionales y ciclos, métodos o funciones, acceso a funciones entre pantallas, y errores en pruebas), y cada tema se puede abrir para ver una explicación y una demostración técnica concreta de ese concepto aplicado dentro de la propia app.

Empecé con una primera versión ("Menú Wearable") con un panel de control que permite cambiar el color de acento (azul, verde, rojo), alternar entre modo claro y oscuro, y ejecutar una prueba try-catch de forma controlada para demostrar el manejo de errores en tiempo real, contando los clics registrados. Después construí una segunda versión más completa ("APA-Menú Wearable Pro"), que agrega buscador de temas, sistema de favoritos (marcar y quitar temas favoritos desde la lista y desde el detalle), más opciones de color (morado, naranja, cian), opciones de accesibilidad (alto contraste, tamaño de texto, modo compacto), un botón para simular un error controlado y un botón para reiniciar todo el sistema a su estado inicial.

- Panel de control con contador de pruebas y cambio de color de acento en tiempo real.
- Prueba try-catch ejecutable desde el propio reloj, con conteo de clics correctos.
- Menú de temas técnicos de la materia, cada uno con su propia pantalla de detalle.
- Sistema de favoritos: marcar y quitar temas favoritos desde la lista y desde el detalle.
- Buscador de temas y opción para limpiar la búsqueda.
- Opciones de accesibilidad: alto contraste, tamaño de texto y modo compacto.
- Botón para simular un error controlado y otro para reiniciar el sistema por completo.

8.5 Código O URL De La Aplicación

El código fuente de este proyecto (Kotlin + Jetpack Compose) vive en un repositorio de GitHub aparte del de PediatrITrack Global, ya que es un proyecto independiente de la materia de Desarrollo para Dispositivos Inteligentes. Este mismo documento se sube a ese repositorio como evidencia de la actividad.

Repositorio Del Proyecto: <https://github.com/AlexAP1115web/APA-AplicacionesWebProgresivas.git>

9. Imágenes De Mi Aplicación

A continuación, documento con capturas de pantalla reales el funcionamiento de ambas partes del proyecto, tanto en el teléfono como en el reloj físico y en los emuladores de Android Studio.

9.1 Alerta Pediátrica: Del Teléfono Al Reloj

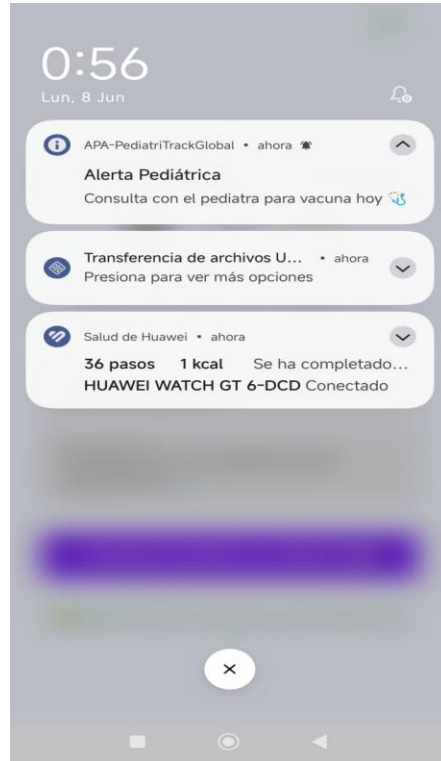


Figura 2. Notificación "Alerta Pediátrica" mostrada en la pantalla de bloqueo del teléfono.



Figura 3. Formulario de la app con el envío exitoso de la alerta hacia el reloj Wear OS.



Figura 4. Caso de error real detectado en pruebas: el reloj Wear OS no fue encontrado.



Figura 5. Reloj físico Huawei Watch GT 6 emparejado, usado como dispositivo real de prueba.

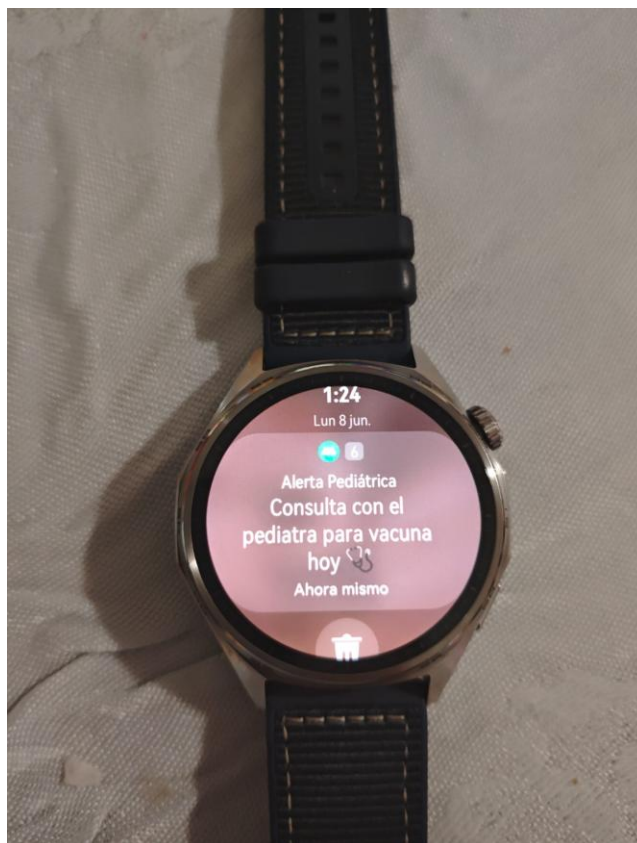


Figura 6. Notificación "Alerta Pediátrica" recibida y visible directamente en el reloj.

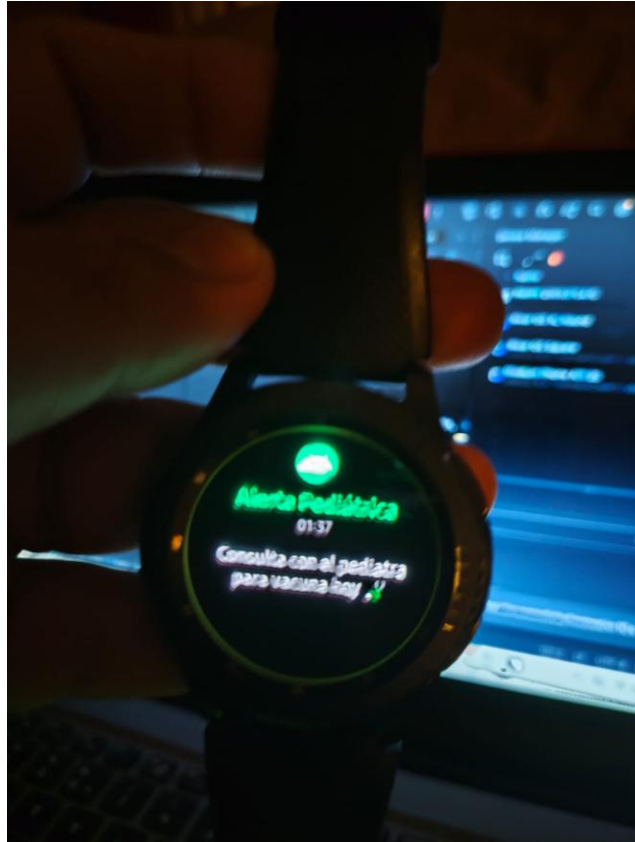


Figura 7. Notificación recibida en el reloj durante una sesión de pruebas en Android Studio.

9.2 Menú Wearable / APA-Menú Wearable Pro: La App Técnica Del Reloj

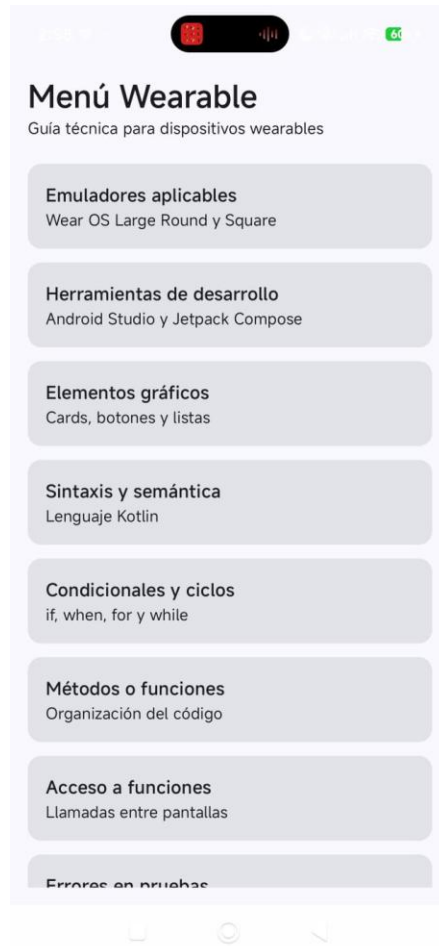


Figura 8. Menú principal con los temas técnicos de la materia (modo claro).

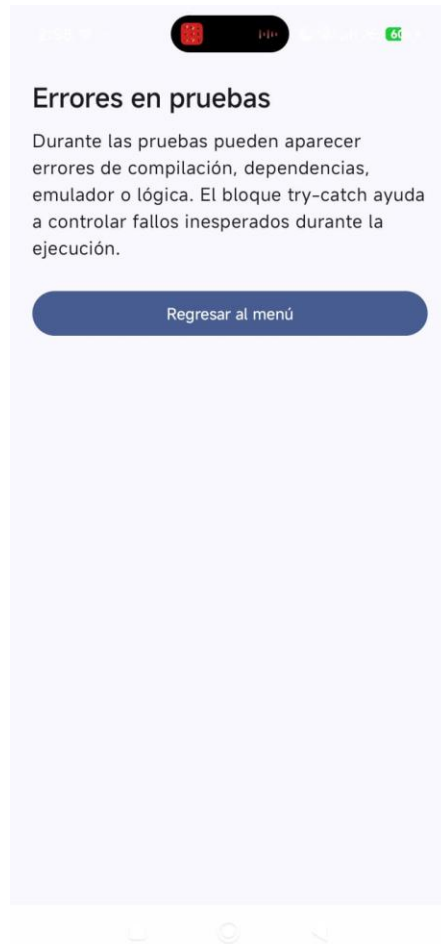


Figura 9. Pantalla de detalle del tema "Errores en pruebas".

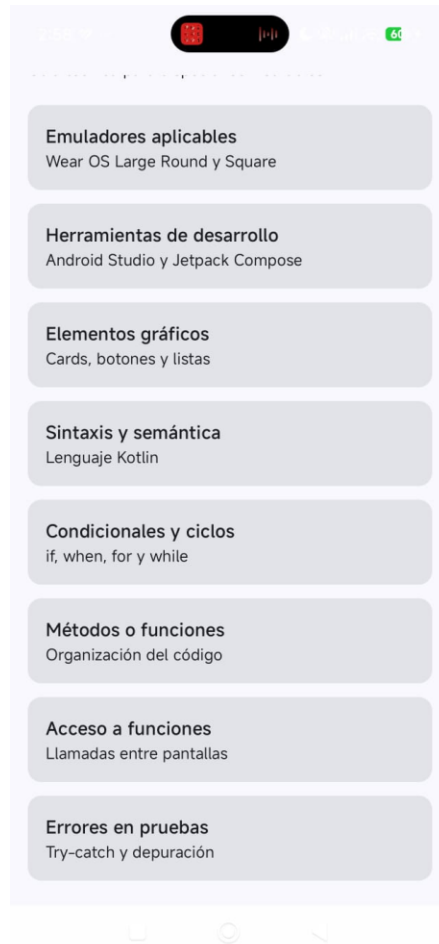


Figura 10. Panel de control en modo oscuro, con acento rojo y contador de clics de prueba.

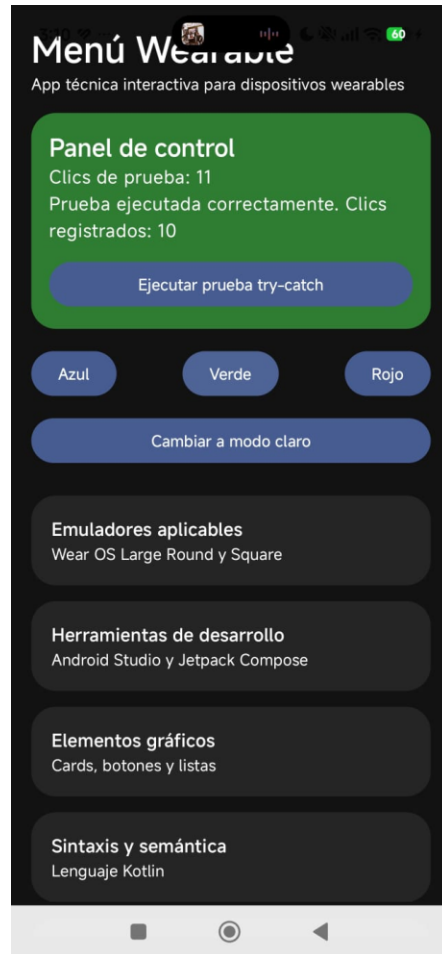


Figura 11. Panel de control con el acento cambiado a color verde.

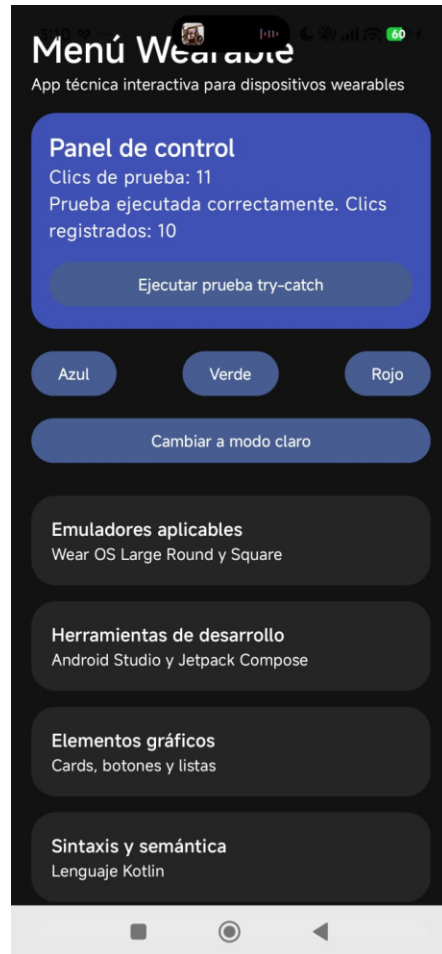


Figura 12. Panel de control con el acento cambiado a color azul/morado.

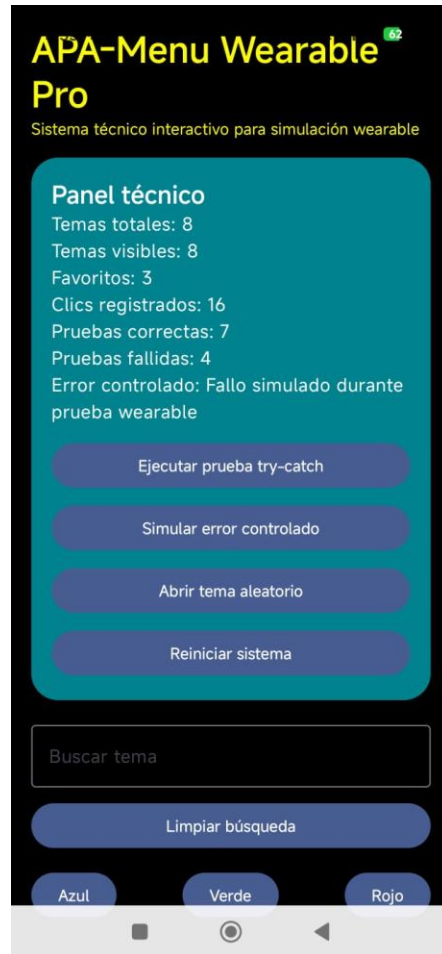


Figura 13. Versión "APA-Menú Wearable Pro": panel técnico con favoritos, pruebas y buscador de temas.



Figura 14. Opciones de accesibilidad: alto contraste, tamaño de texto, modo compacto y favoritos.



Figura 15. Lista de temas técnicos con el estado de favorito marcado en cada tarjeta.

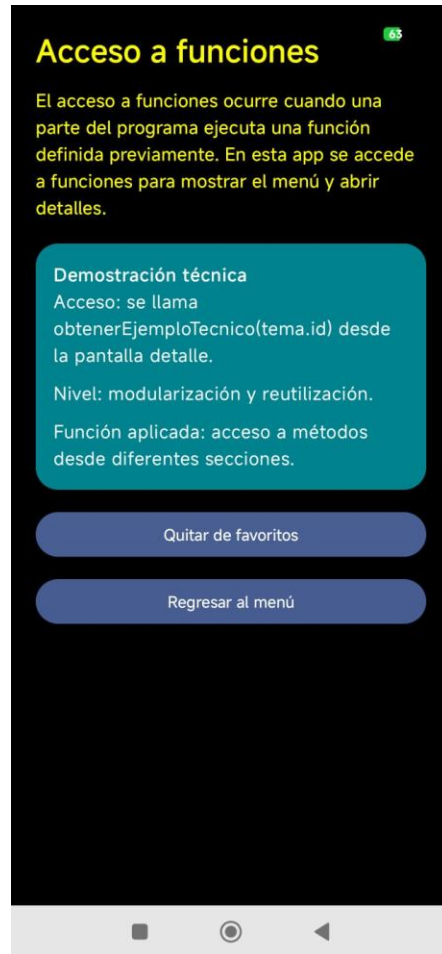


Figura 16. Detalle del tema "Acceso a funciones", con favorito activado.



Figura 17. Detalle del tema "Herramientas de desarrollo": Android Studio y Jetpack Compose.

10. Ejemplo De Una Aplicación Nativa Reconocida: Spotify

Además de mi propio proyecto, documento también el ejemplo clásico de aplicación nativa a nivel industria que se suele usar para explicar el concepto: Spotify. Cuenta con dos versiones de código completamente independientes: una escrita en Swift para iOS y otra en Kotlin para Android, cada una integrada con el framework de interfaz oficial de su plataforma (SwiftUI/UIKit y Jetpack Compose, respectivamente).

Gracias a ser una aplicación nativa, Spotify puede reproducir audio en segundo plano mientras el usuario utiliza otras aplicaciones, descargar canciones y reproducirlas sin conexión a internet (almacenándolas cifradas en el almacenamiento local del dispositivo), integrarse con los controles nativos de reproducción del sistema operativo (pantalla de bloqueo, centro de control, Android Auto y CarPlay) y enviar notificaciones push para recomendaciones y novedades. Ninguna de estas funciones —en particular la reproducción en segundo plano y el acceso a controles del sistema— podría implementarse con el mismo nivel de integración mediante una aplicación web, ya que requieren acceso directo a las APIs del sistema operativo que solo el desarrollo nativo permite.

App Store (iOS): <https://apps.apple.com/app/spotify-music-and-podcasts/id324684580>

Google Play (Android): <https://play.google.com/store/apps/details?id=com.spotify.music>

Sitio web oficial: <https://www.spotify.com>

11. Lista de Cotejo

Reviso, a manera de autoevaluación, que el documento cumpla con todos los puntos solicitados para esta actividad:

ELEMENTO SOLICITADO	CUMPLE	OBS.
Portada con formato oficial UTP	Sí	
Índice / tabla de contenido	Sí	
Introducción	Sí	
Información / definición de qué es una aplicación nativa	Sí	
Ilustración del funcionamiento (diagrama)	Sí	
Características principales	Sí	
Ventajas y desventajas	Sí	
Tabla comparativa (nativa vs. web vs. híbrida)	Sí	
Tecnología utilizada (lenguaje, IDE, framework)	Sí	
Ejemplo de mi propia aplicación nativa (proyecto Wear OS)	Sí	
Imágenes de mi aplicación (evidencia propia)	Sí	
Código o URL de mi aplicación	Sí	
Ejemplo de aplicación nativa reconocida en la industria (Spotify)	Sí	
URL de la aplicación de referencia	Sí	
Conclusión	Sí	
Referencias en formato APA (incluyendo uso de IA)	Sí	

Tabla Comparativa De Aplicación Nativa Vs Mi Propuesta

CRITERIO DE APP NATIVA	QUÉ EXIGE LA DEFINICIÓN	¿CUMPLE?	EVIDENCIA EN MI PROYECTO
Lenguaje y SDK oficial	Debe escribirse con el lenguaje y SDK oficial del sistema, no HTML/JS	Sí	Escrito 100% en Kotlin, con el SDK oficial de Wear OS de Google
IDE oficial	Debe construirse con el entorno oficial de la plataforma	Sí	Desarrollado íntegramente en Android Studio
Framework de interfaz nativo	La interfaz debe usar componentes nativos, no un WebView	Sí	Toda la interfaz en Jetpack Compose para Wear OS
Instalación vía compilado nativo	Debe compilarse a un paquete instalable en el dispositivo	Sí	Se compila e instala en el emulador Wear OS y en el Huawei Watch GT 6
Acceso completo al hardware	Debe poder usar hardware/funciones del sistema	Sí	API de notificaciones + Bluetooth entre teléfono y reloj (Alerta Pediátrica)
Funciona sin internet	Las funciones clave no deben depender de internet	Sí	Menú Wearable y panel de control corren toda su lógica localmente
Rendimiento nativo sin capas intermedias	Debe ejecutarse directo sobre el SO, sin motor web	Sí	Sin WebView; Compose dibuja directo sobre Wear OS
Código independiente por plataforma	Requiere base de código propia por plataforma	Sí	Proyecto Kotlin separado de PediatriTrack Global (React Native)

12. Conclusión

Las aplicaciones nativas representan la opción de desarrollo móvil con mejor rendimiento y mayor capacidad de integración con el hardware y el sistema operativo del dispositivo, a cambio de requerir un código fuente independiente para cada plataforma, lo que incrementa el tiempo y el costo de desarrollo en comparación con soluciones web o híbridas. Como se observa en la comparativa de la sección 6, la elección entre nativo, web e híbrido depende de qué tanto pesen el rendimiento y el acceso al hardware frente al costo y tiempo de desarrollo en cada proyecto.

Para aplicaciones donde el rendimiento, el uso intensivo de hardware o el funcionamiento sin conexión son críticos para la experiencia del usuario —como se ve en el caso de Spotify, y como comprobé de primera mano al construir mi propio proyecto de Wear OS—, el desarrollo nativo continúa siendo la opción más adecuada. Programar directamente para el reloj me dejó mucho más claro por qué el acceso al hardware (en este caso, notificaciones sincronizadas por Bluetooth entre teléfono y reloj) es algo que una app web simplemente no puede replicar. Comprender estas ventajas y limitaciones me permite, como desarrollador, elegir el enfoque más conveniente según las necesidades específicas de cada proyecto.

13. Referencias

- Actualizatec. (s. f.). *¿Qué es una app nativa? Características y ventajas*. <https://actualizatec.com/blog/app-nativa/>
- Anthropic. (2026). *Claude (Sonnet 4.5)* [Modelo de lenguaje de gran tamaño]. <https://claude.ai>
- Bambu Mobile. (s. f.). *¿Qué es una app nativa? Características y usos*. <https://bambu-mobile.com/app-nativa-que-es-caracteristicas-y-usos/>
- Dribba. (2024). *¿Qué es una aplicación nativa? Ventajas y cuándo usarla*. <https://www.dribba.com/post/que-es-una-aplicaci%C3%B3n-nativa-te-lo-contamos-todo-en-2024>
- Fonzi. (s. f.). *What is a native app? Definition, examples, and pros and cons*. <https://fonzi.ai/blog/what-is-a-native-app>
- Google for Developers. (s. f.). *Jetpack Compose for Wear OS overview*. Android Developers. <https://developer.android.com/training/wearables/compose>
- Maddevs. (s. f.). *Native app: Meaning, examples, and native vs hybrid apps*. <https://maddevs.io/glossary/native-app/>
- Spotify AB. (s. f.). *Spotify: Music and podcasts* [Aplicación móvil]. App Store. <https://apps.apple.com/app/spotify-music-and-podcasts/id324684580>
- Spotify AB. (s. f.). *Spotify: Music and podcasts* [Aplicación móvil]. Google Play. <https://play.google.com/store/apps/details?id=com.spotify.music>